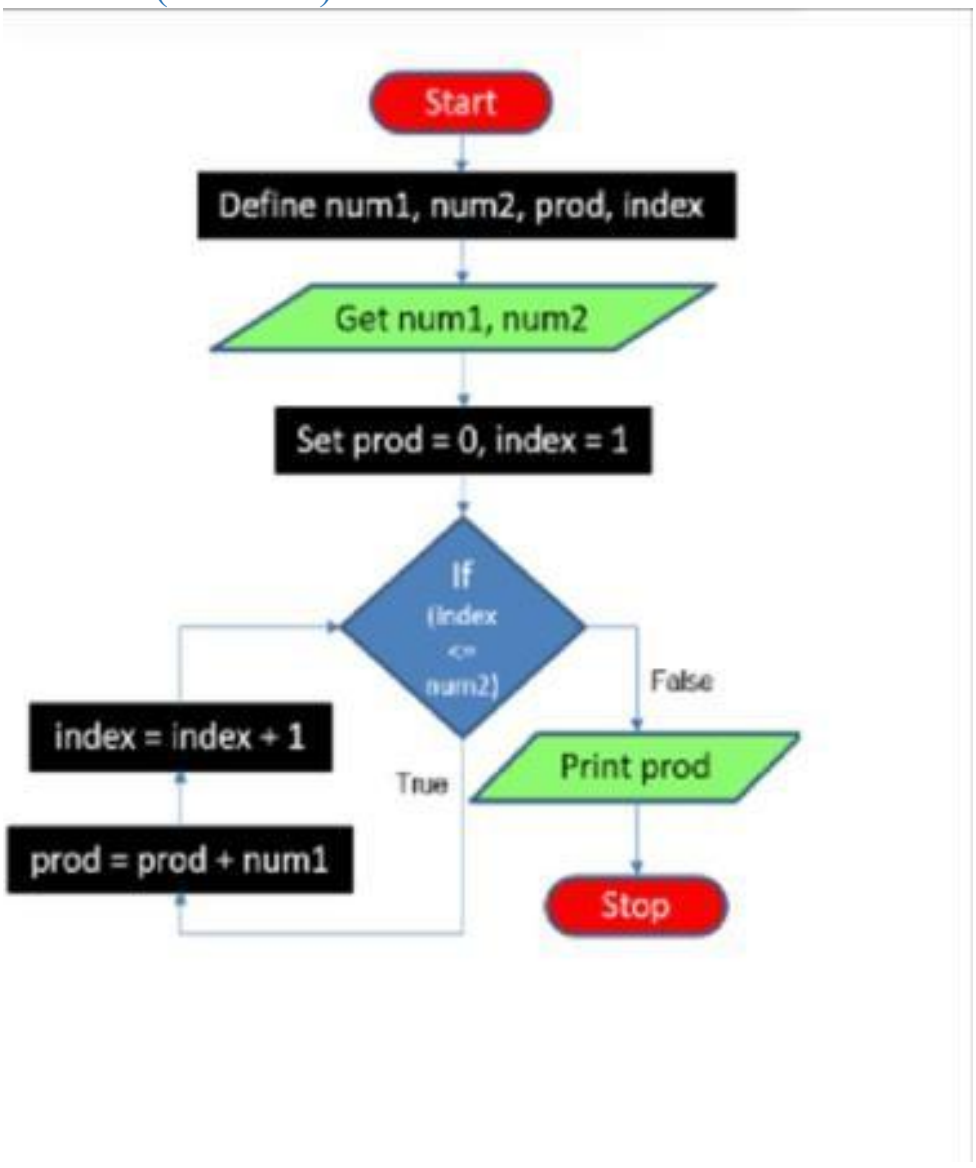
1. Problem Statement

Find the product of two integers without using the multiplication operator (*), using the logic of repeated addition. The solution must correctly handle all sign combinations (positive × positive, positive × negative, negative × positive, negative × negative) and multiplication by zero, while using the minimum possible number of additions.

2. Given Algorithm (Incorrect)

```
Define num1, num2, prod, index as integers
Get num1, num2
Set prod = 0, index = 1
Repeat steps 4.1 and 4.2 while (index <= num2):
     4.1  prod = prod + num1
     4.2  index = index + 1
Print prod
Stop
```

3. Given Flowchart (Incorrect)



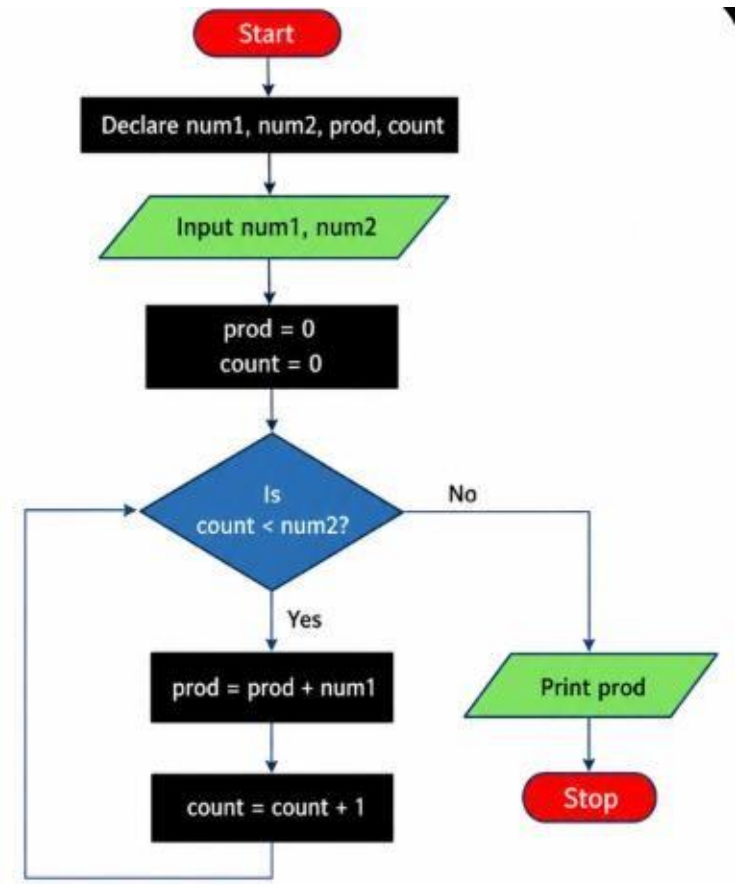
4. Mistakes Identified

1. **Non-standard initialization:** index starts from **1**. Although this works with the condition index <= num2, starting the counter from **0** is clearer and more conventional.
2. **Less clear loop condition:** The condition index <= num2 works only because index starts at 1. A cleaner combination is:
   count = 0 and count < num2.
3. **Order of operations is confusing:** In the diagram, index = index + 1 is shown before prod = prod + num1 in the loop layout. It is clearer to first add num1 to prod and then increment the counter.
4. **Does not handle negative num2:** If num2 is negative, the condition becomes false immediately and the program incorrectly prints 0.
5. **Input assumption is not mentioned:** The given algorithm is suitable only when num2 is a **non-negative integer**.
6. **Flowchart readability:** The loop arrows and placement of the increment/addition blocks make the execution sequence harder to understand. The corrected flowchart has a straightforward sequence:

5. Corrected Algorithm

1. Define num1, num2, prod, index, sign, count, addend as integers.
2. Get num1, num2.
3. If num1 = 0 OR num2 = 0, set prod = 0 and go to Step 12.
4. If num1 < 0 AND num2 < 0, set sign = 1 (negative × negative gives a positive answer).
5. Otherwise, if num1 < 0 OR num2 < 0, set sign = -1 (exactly one operand is negative).
6. Otherwise (both positive), set sign = 1.
7. If num1 < 0, set num1 = -num1. If num2 < 0, set num2 = -num2 (this makes both numbers positive).
8. If num1 < num2, set count = num1 and addend = num2; otherwise set count = num2 and addend = num1 (using the smaller value as the loop counter guarantees the minimum number of additions).
9. Set prod = 0, index = 1.
10. Repeat while (index <= count): prod = prod + addend; index = index + 1.
11. If sign = -1, set prod = -prod.
12. Print prod.
13. Stop.

6. Corrected Flowchart



7. Verification Against Given Test Cases

The corrected C program was compiled and executed against all 13 test cases supplied in the assignment. Every actual output matched the expected output and used exactly the minimum number of additions specified.

TC#	Input (num1, num2)	Expected Product	Actual Product	Min. Additions (Expected)	Actual Additions	Result
1	12, 3	36	36	3	3	Pass
2	3, 12	36	36	3	3	Pass
3	-6, 5	-30	-30	5	5	Pass
4	6, -5	-30	-30	5	5	Pass
5	-9, -4	36	36	4	4	Pass
6	-4, -9	36	36	4	4	Pass
7	0, 8	0	0	0	0	Pass
8	8, 0	0	0	0	0	Pass
9	0, -7	0	0	0	0	Pass
10	-7, 0	0	0	0	0	Pass
11	0, 0	0	0	0	0	Pass
12	1, -7	-7	-7	1	1	Pass
13	-1, -9	9	9	1	1	Pass

8. Conclusion

The given algorithm's core flaw is that its loop bound, "index <= num2", is only ever satisfied when num2 is positive; a negative num2 makes the loop skip entirely and silently returns 0. It also does not minimize additions, since it always iterates num2 times regardless of which operand is smaller. The corrected version fixes both issues: it works out the sign of the answer with simple if-else checks, converts both numbers to positive, and always loops the smaller of the two magnitudes while adding the larger one.

Name- Keshav Mittal

Batch-16

SAP ID- 590032934